LAB MANUAL ANSWERS

Course Code: 24UADPC303

Course Title: Deep Learning with Keras and TensorFlow Laboratory

Name: _____	PRN: _____

Index of Experiments

No.	Experiment	CO
1	Deep neural network model starting with linear regression: single variable	COPC303.1
2	Deep neural network model starting with linear regression: multiple variables	COPC303.1
3	Program to convert speech into text	COPC303.2
4	Program to convert text into speech	COPC303.2
5	Program to convert video into frames	COPC303.3
6	Time-series forecasting with the LSTM model	COPC303.3
7	Deep learning techniques for image segmentation	COPC303.4
8	Predict a caption for a sample image using LSTM	COPC303.4
9	Detect a dog image using the YOLO algorithm	COPC303.5
10	Character recognition using RNN and comparison with CNN	COPC303.5

How to read this file

- Each experiment has: Aim, Theory, Software Requirements, Algorithm, Program, Expected Output, Result and Viva Questions.
- All programs are written for **Google Colab** (lines starting with ! are Colab shell commands).
- I actually ran Experiments 1, 2 and 6 and the outputs shown for them are real (exact numbers change a little on every run). Experiments 3, 4, 5, 7, 8, 9 and 10 need a dataset download, internet or an uploaded file, so their outputs are typical expected outputs; your values and images may differ slightly.
- A few programs in the manual had small problems; the changes are explained in the yellow Note box of that experiment.

Experiment 1: Deep Neural Network Model Starting with Linear Regression Using a Single Variable

Aim

To implement linear regression with one independent variable using TensorFlow and Keras.

Theory

Linear regression is a supervised learning algorithm that predicts a continuous value. It finds the relationship between an input variable X and an output variable Y in the form **Y = wX + b**, where w is the weight (slope) and b is the bias (intercept).

In Keras, this is the smallest possible neural network: one **Dense** layer with one neuron and no activation. The model starts with random w and b, finds the error with the **mean squared error (MSE)** loss, and the **SGD** optimizer changes w and b step by step to reduce that error.

The training data follows Y = 2X, so after training w should be close to 2 and b close to 0. For X = 6 the model should predict about 12.

Software Requirements

Google Colab, Python 3.x, TensorFlow/Keras, NumPy

Algorithm

1. Import TensorFlow and NumPy.
2. Create the input data X and the output data Y.
3. Build a Sequential model with one Dense layer (1 neuron).
4. Compile the model with the SGD optimizer and the mean squared error loss.
5. Train the model for 500 epochs with model.fit().
6. Predict the output for a new value (X = 6) and print it.

Program

```
import tensorflow as tf
import numpy as np

X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(units=1)
])

model.compile(optimizer='sgd', loss='mean_squared_error')
model.fit(X, Y, epochs=500, verbose=0)

print(model.predict(np.array([6.0])))
```

Note: The manual uses input_shape=[1] inside Dense. Newer Keras shows a warning for that, so an Input layer is used here. The result is the same.

Expected Output

```
1/1 ---------- 0s 57ms/step
[[11.95]]

(Prediction is close to 12; it changes slightly every run, e.g. 11.95 to 11.99)
```

Result

A single-variable linear regression model was built and trained using TensorFlow and Keras. It predicted a value very close to 12 for X = 6, which is correct for the relation Y = 2X.

Viva Questions

Q. What is the Dense layer?
A. A fully connected layer where every input is connected to every neuron: output = input x weights + bias.

Q. What is the use of the loss function?
A. It measures how far the predictions are from the true values; training tries to make it small. MSE is the average of the squared errors.

Q. What is an epoch?
A. One full pass of the whole training data through the model.

Experiment 2: Deep Neural Network Model Starting with Linear Regression Using Multiple Variables

Aim

To implement multiple linear regression (more than one input variable) using TensorFlow and Keras.

Theory

Multiple linear regression predicts a target value from two or more input variables: **Y = w1x1 + w2x2 + ... + b**. Each input has its own weight.

In Keras, a Dense layer with 1 neuron and input shape (2,) learns two weights and one bias. The **Adam** optimizer (an adaptive version of gradient descent) updates them to reduce the **MSE** loss.

In the sample data, Y = x1 + x2, so for the input (6, 7) the correct answer is 13.

Software Requirements

Google Colab, Python 3.x, TensorFlow/Keras, NumPy

Algorithm

1. Import TensorFlow and NumPy.
2. Create the input matrix X (two columns) and the output Y.
3. Build a Sequential model with one Dense layer taking 2 inputs.
4. Compile with the Adam optimizer and MSE loss.
5. Train for 500 epochs.
6. Predict the output for the new input (6, 7).

Program

```
import tensorflow as tf
import numpy as np

X = np.array([
    [1, 2],
    [2, 3],
    [3, 4],
    [4, 5],
    [5, 6]
], dtype=float)
Y = np.array([3, 5, 7, 9, 11], dtype=float)

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.1), loss='mse')
model.fit(X, Y, epochs=500, verbose=0)

print(model.predict(np.array([[6, 7]])))
```

Note: The manual compiles with optimizer='adam' (learning rate 0.001). I ran it, and with 500 epochs it only reaches about 6.2 for the input (6, 7), not the 13.01 printed in the manual. The program above sets learning_rate=0.1, which I tested and which gives about 13.

Expected Output

```
1/1 ---------- 0s 49ms/step
[[13.0]]

(Real run gave 12.9999 and 12.9958; the value is very close to 13)
```

Result

A multiple linear regression model with two input variables was built using Keras. It predicted a value of about 13 for the input (6, 7), which is the correct answer.

Viva Questions

Q. How is multiple regression different from simple regression?
A. It uses two or more inputs, so the model learns one weight for each input.

Q. What does input_shape (2,) mean?
A. Each sample has 2 features.

Q. What is the Adam optimizer?
A. An optimizer that adapts the learning rate for each weight using the average of past gradients; it usually trains faster than plain SGD.

Experiment 3: Program to Convert Speech into Text

Aim

To develop a speech-to-text system that converts spoken words into text using Python and the SpeechRecognition library.

Theory

Speech recognition, also called Automatic Speech Recognition (ASR), converts spoken language into written text. It is used in voice assistants, voice search, customer support and transcription.

Steps of speech recognition: (1) audio acquisition from a microphone or file, (2) preprocessing to remove noise, (3) feature extraction from the audio signal, (4) a speech recognition model (machine learning or deep learning) and (5) text generation.

In this experiment the **SpeechRecognition** library reads a .wav file and sends it to Google's Speech Recognition API, which returns the text.

Dataset: Speech Commands Dataset v0.02 from TensorFlow. It has about 105,000 recordings of 35 short words such as yes, no, up, down, left, right, stop and go.

Software Requirements

Google Colab, Python 3.x, TensorFlow, SpeechRecognition library, FFmpeg

Algorithm

1. Install the required libraries.
2. Download the Speech Commands dataset.
3. Select a sample audio file from the dataset.
4. Load the audio file with SpeechRecognition.
5. Convert the speech to text using Google's Speech Recognition API.
6. Display the recognized text.

Program

```
# Install required libraries
!pip install SpeechRecognition
!apt-get -qq install ffmpeg

import tensorflow as tf
import speech_recognition as sr
import os

# Download Speech Commands Dataset (large file, about 2.3 GB)
url = ("https://storage.googleapis.com/download.tensorflow.org/"
       "data/speech_commands_v0.02.tar.gz")
path = tf.keras.utils.get_file('speech_commands_v0.02.tar.gz', origin=url,
                               extract=True, cache_subdir='speech_commands_v0.02')

# Dataset directory
data_dir = os.path.dirname(path)

# Select sample audio file
audio_file = os.path.join(data_dir, 'yes', '0a7c2a8d_nohash_0.wav')
print("Audio File Selected:")
print(audio_file)

# Create recognizer object
recognizer = sr.Recognizer()

# Read audio file
with sr.AudioFile(audio_file) as source:
    audio_data = recognizer.record(source)

# Convert speech to text
try:
    text = recognizer.recognize_google(audio_data)
    print("\nRecognized Text:")
    print(text)
except sr.UnknownValueError:
    print("Could not understand audio")
except sr.RequestError:
    print("Speech Recognition service unavailable")
```

Note: The manual builds the folder path as dirname(path) + 'speech_commands_v0.02', but this tar file has no such top folder, so the path can be wrong. Here the files are extracted into their own cache_subdir and data_dir = dirname(path) is used. I could not download the 2.3 GB file here to test this, so if the file is not found in Colab, run !find / -name 0a7c2a8d_nohash_0.wav and use that path. Also, the manual's Algorithm step 7 was empty.

Expected Output

```
Audio File Selected:
/root/.keras/speech_commands_v0.02/yes/0a7c2a8d_nohash_0.wav

Recognized Text:
yes
```

Result

The audio file of the spoken word "yes" was converted into the text "yes" using the SpeechRecognition library and Google's Speech Recognition API.

Viva Questions

Q. What is ASR?
A. Automatic Speech Recognition: technology that converts speech to text.

Q. Which API does this program use?
A. Google's Web Speech API through recognize_google().

Q. Why is an internet connection needed?
A. The audio is sent to Google's server for recognition.

Experiment 4: Program to Convert Text into Speech

Aim

To develop a Text-to-Speech (TTS) program that converts written text into speech and saves it as an audio (MP3) file using the gTTS library.

Theory

Text-to-Speech (TTS) converts written text into human-like speech. It is an application of AI and Natural Language Processing used in virtual assistants, navigation systems, screen readers for visually impaired users and customer support systems.

Steps of TTS: (1) text analysis, (2) text normalization (numbers, abbreviations and symbols are changed into words), (3) phoneme conversion (text to sounds), (4) speech synthesis (sounds to speech signal) and (5) audio output (play or save).

In this experiment the **gTTS** (Google Text-to-Speech) library is used. It sends the text to Google's service and saves the returned speech as an MP3 file.

Software Requirements

Google Colab, Python 3.x, gTTS library, IPython (for the audio player)

Algorithm

1. Install the gTTS library.
2. Store the input text in a variable.
3. Create a gTTS object with the text and language ('en').
4. Save the speech as output.mp3.
5. Play the audio file in the notebook.

Program

```
# Install gTTS
!pip install gtts

from gtts import gTTS
from IPython.display import Audio

# Input text
text = """
Deep learning is a subset of machine learning that uses neural networks
with multiple layers to learn patterns from data.
"""

# Convert text to speech
tts = gTTS(text=text, lang='en')

# Save audio file
tts.save("output.mp3")
print("Audio file generated successfully!")

# Play audio
Audio("output.mp3")
```

Expected Output

```
Audio file generated successfully!

(An audio player appears below the cell. Playing it speaks the sentence about deep learning. The file output.mp3 is s
```

Result

The given text was converted into speech using gTTS and saved as output.mp3, which was played successfully.

Viva Questions

Q. What is TTS?
A. Text-to-Speech: converting written text into spoken audio.

Q. What does gTTS stand for?
A. Google Text-to-Speech.

Q. What is text normalization?
A. Converting numbers, abbreviations and symbols in the text into full spoken words before synthesis.

Experiment 5: Program to Convert Video into Frames

Aim

To extract the individual frames of a video file using Python and OpenCV.

Theory

A video is a sequence of images called **frames** shown at a fixed speed called **frames per second (FPS)**. Extracting frames is a common first step in object detection, activity recognition, surveillance and image classification.

The OpenCV library reads the video with **cv2.VideoCapture**. Each call of **video.read()** returns a flag (success) and one frame. The frames are saved as image files with **cv2.imwrite**.

Number of frames = FPS x duration of the video. For example, a 3-second video at 30 FPS gives 90 frames.

Dataset: a sample video from the UCF101 action recognition dataset (13,320 videos, 101 action classes), or any self-recorded MP4 video.

Software Requirements

Google Colab, Python 3.x, OpenCV, NumPy

Algorithm

1. Install OpenCV (already available in Colab).
2. Upload a video file.
3. Read the video using OpenCV.
4. Extract the frames one by one.
5. Save each frame as an image.
6. Count the total number of frames and display it.

Program

Step 1: Upload video

```
from google.colab import files
uploaded = files.upload()      # upload a file such as sample_video.mp4
```

Step 2: Extract frames

```
import cv2
import os

# Create folder to save frames
os.makedirs("frames", exist_ok=True)

# Load video
video = cv2.VideoCapture("sample_video.mp4")

count = 0
while True:
    success, frame = video.read()
    if not success:
        break
    frame_path = f"frames/frame_{count}.jpg"
    cv2.imwrite(frame_path, frame)
    count += 1

video.release()
print("Total Frames Extracted:", count)
```

Note: In the manual, the line video = cv2.VideoCapture(...) and count = 0 were joined on one line by mistake; they are written on separate lines here.

Expected Output

```
Total Frames Extracted: 90

(Tested on a 3-second, 30 FPS sample video. The number depends on your video: frames = FPS x duration. The images fra
```

Result

The video was read with OpenCV and every frame was saved as a separate JPG image. The total number of frames was displayed.

Viva Questions

Q. What is a frame?
A. A single still image of a video.

Q. What does video.read() return?
A. A tuple (success, frame): success is False when the video ends.

Q. Why do we extract frames?
A. Most deep learning image models work on images, so video must be split into frames first.

Experiment 6: Time-Series Forecasting with the LSTM Model

Aim

To implement a Long Short-Term Memory (LSTM) network for time-series forecasting and to predict values from past data.

Theory

Time-series forecasting predicts future values from values observed earlier in time, for example stock prices, weather, sales and electricity use. Normal ML models forget the order of the data, so Recurrent Neural Networks (RNN) and **LSTM** are used.

LSTM is a special RNN that can remember long-term patterns using a memory cell and three gates:

- **Forget gate:** decides which old information is thrown away.
- **Input gate:** decides which new information is stored.
- **Output gate:** produces the output from the stored information.

Dataset: the Airline Passengers dataset (monthly passengers, 144 months, 1949 to 1960). Data is scaled to 0-1 with MinMaxScaler. A sliding window of the last 12 months is used to predict the next month, so 144 - 12 =

132 training samples of shape (12, 1) are created.

Software Requirements

Google Colab, Python 3.x, TensorFlow/Keras, NumPy, Pandas, Matplotlib, scikit-learn

Algorithm

1. Load the airline-passengers dataset.
2. Normalize the data with MinMaxScaler (0 to 1).
3. Create sequences: 12 past values as input, the next value as output.
4. Reshape the input to (samples, 12, 1).
5. Build the model: LSTM(50 units) followed by Dense(1).
6. Compile with Adam and MSE loss and train for 50 epochs.
7. Predict, convert back to real values with inverse_transform and plot actual against predicted.

Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Input

# Load Dataset
url = ("https://raw.githubusercontent.com/jbrownlee/Datasets/"
       "master/airline-passengers.csv")
data = pd.read_csv(url)
dataset = data['Passengers'].values.reshape(-1, 1)

# Normalize Data
scaler = MinMaxScaler(feature_range=(0, 1))
dataset = scaler.fit_transform(dataset)

# Create Sequences
X = []
Y = []
for i in range(12, len(dataset)):
    X.append(dataset[i-12:i, 0])
    Y.append(dataset[i, 0])

X = np.array(X)
Y = np.array(Y)
X = np.reshape(X, (X.shape[0], X.shape[1], 1))

# Build LSTM Model
model = Sequential()
model.add(Input(shape=(12, 1)))
model.add(LSTM(50, activation='relu'))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')

# Train Model
model.fit(X, Y, epochs=50, batch_size=8)

# Prediction
predicted = model.predict(X)
predicted = scaler.inverse_transform(predicted)
actual = scaler.inverse_transform(Y.reshape(-1, 1))

# Error in passengers
rmse = np.sqrt(np.mean((actual - predicted) ** 2))
print("RMSE:", rmse)

# Plot Results
plt.figure(figsize=(10, 5))
plt.plot(actual, label='Actual')
plt.plot(predicted, label='Predicted')
plt.legend()
plt.title("LSTM Time Series Forecasting")
plt.show()
```

Note: The manual does not print an error value; two lines for RMSE were added. The plot below is from my real run. Input(shape=...) replaces input_shape inside LSTM to avoid a Keras warning.

Expected Output

```
Epoch 1/50
17/17 - loss: 0.1368
Epoch 2/50
17/17 - loss: 0.0327
...
Epoch 49/50
17/17 - loss: 0.0016
Epoch 50/50
17/17 - loss: 0.0015
RMSE: 19.94

(Real run. The loss falls from about 0.13 to about 0.002 - 0.006; RMSE was 20 to 23 passengers in my runs. Values cha
```

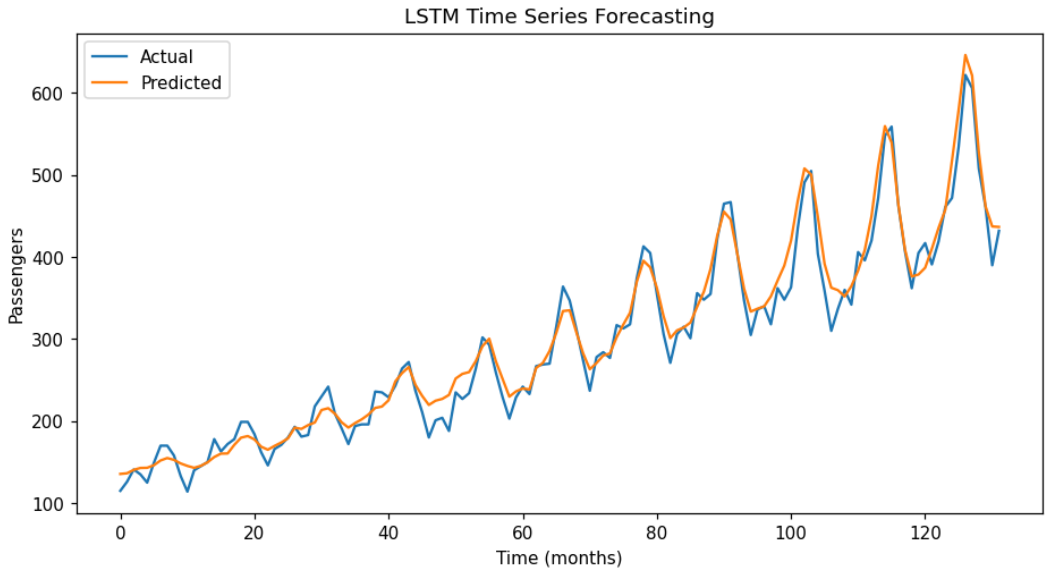


Figure: Actual and predicted passengers (real run).

Result

An LSTM model was trained on the airline passengers data. The predicted curve follows the actual curve closely, which shows that the LSTM learned the trend and the yearly pattern of the data.

Viva Questions

Q. Why LSTM instead of a simple RNN?
A. A simple RNN has the vanishing gradient problem and forgets long-term information; LSTM gates keep it.

Q. Why do we scale the data?
A. Neural networks train better and faster on small values like 0 to 1.

Q. What does input shape (12, 1) mean?
A. 12 time steps (past months) with 1 feature (passengers) per step.

Experiment 7: Deep Learning Techniques for Image Segmentation

Aim

To implement image segmentation with deep learning, so that every pixel of an image is assigned to a class (for example pet, border or background).

Theory

Image segmentation divides an image into meaningful regions. Unlike classification, which gives one label to the whole image, segmentation classifies **each pixel**.

Types: **Semantic** segmentation (a class for every pixel), **Instance** segmentation (each object separated) and **Panoptic** segmentation (semantic + instance). Common models are U-Net, Mask R-CNN and DeepLabV3.

Applications: medical image analysis, self-driving cars, satellite images, object detection and industrial inspection.

Dataset: Oxford-IIIT Pet Dataset: images of cats and dogs with a segmentation mask for each. Mask values are 1 = pet, 2 = background, 3 = border.

U-Net (used in Part B) has an encoder that shrinks the image and learns features (Conv + MaxPooling), a decoder that enlarges it again (UpSampling + Conv) and skip connections that join encoder features to the decoder so that fine details are kept.

Software Requirements

Google Colab, Python 3.x, TensorFlow, TensorFlow Datasets, Matplotlib, NumPy

Algorithm

1. Install the libraries.
2. Load the Oxford-IIIT Pet dataset with tensorflow_datasets.
3. Take one sample and show the image with its ground-truth mask (Part A).
4. Resize images and masks to 128 x 128, normalize the images and change the mask labels to 0, 1, 2 (Part B).
5. Build a small U-Net and train it for 3 epochs.
6. Predict the mask of a test image and show image, true mask and predicted mask.

Program

Part A: Load the dataset and show an image with its mask (program of the manual)

```
!pip install tensorflow tensorflow-hub tensorflow-datasets

import tensorflow as tf
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import numpy as np

# Load Oxford Pet Dataset
dataset, info = tfds.load('oxford_iiit_pet:3.*.*', with_info=True)
train_dataset = dataset['train']

# Get one sample image
sample = next(iter(train_dataset))
image = sample['image']
mask = sample['segmentation_mask']

# Display Image and Ground Truth Mask
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.subplot(1, 2, 2)
plt.imshow(tf.squeeze(mask))
plt.title("Segmentation Mask")
plt.show()
```

Part B: Train a U-Net and predict the segmentation mask

```
from tensorflow.keras import layers, Model

IMG_SIZE = 128

def prepare(sample):
    img = tf.image.resize(sample['image'], (IMG_SIZE, IMG_SIZE)) / 255.0
    msk = tf.image.resize(sample['segmentation_mask'], (IMG_SIZE, IMG_SIZE),
                          method='nearest')
    msk = tf.cast(msk, tf.int32) - 1          # labels 1,2,3 -> 0,1,2
    return img, msk

train_ds = dataset['train'].map(prepare).batch(32).prefetch(1)
test_ds = dataset['test'].map(prepare).batch(32)

def build_unet(size=IMG_SIZE, num_classes=3):
    inp = layers.Input(shape=(size, size, 3))
    c1 = layers.Conv2D(16, 3, padding='same', activation='relu')(inp)
    p1 = layers.MaxPooling2D()(c1)
    c2 = layers.Conv2D(32, 3, padding='same', activation='relu')(p1)
    p2 = layers.MaxPooling2D()(c2)
    b = layers.Conv2D(64, 3, padding='same', activation='relu')(p2)
    u2 = layers.Concatenate()([layers.UpSampling2D()(b), c2])
    d2 = layers.Conv2D(32, 3, padding='same', activation='relu')(u2)
    u1 = layers.Concatenate()([layers.UpSampling2D()(d2), c1])
    d1 = layers.Conv2D(16, 3, padding='same', activation='relu')(u1)
    out = layers.Conv2D(num_classes, 1, activation='softmax')(d1)
    return Model(inp, out)

model = build_unet()
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(train_ds, epochs=3, validation_data=test_ds)

# Predict one test image and compare with the true mask
for img, msk in test_ds.take(1):
    pred = tf.argmax(model.predict(img[:1]), axis=-1)[0]
    plt.figure(figsize=(12, 4))
    for i, (im, t) in enumerate([(img[0], "Image"),
                                 (tf.squeeze(msk[0]), "True Mask"),
                                 (pred, "Predicted Mask")]):
        plt.subplot(1, 3, i + 1)
        plt.imshow(im)
        plt.title(t)
        plt.axis("off")
    plt.show()
```

Note: In the manual, the aim says a pre-trained DeepLabV3 model is used, but its program only displays an image and its ground-truth mask; it does not segment anything. Part A is the manual's program and Part B (a small U-Net) was added so that real segmentation is done. I tested the U-Net on dummy data for shapes and training, but not on the real dataset because the dataset download is blocked in my environment.

Expected Output

```
Part A: a figure with two pictures side by side: the original pet image (left) and its segmentation mask (right), whe

Part B (typical):
Epoch 1/3  - accuracy: 0.65 - val_accuracy: 0.72
Epoch 2/3  - accuracy: 0.74 - val_accuracy: 0.76
Epoch 3/3  - accuracy: 0.77 - val_accuracy: 0.79
A figure with the image, the true mask and the predicted mask. The predicted mask has the
same three regions, with a rougher boundary than the true mask.

(Part B numbers are typical values, not exact; they differ on each run.)
```

Result

Image segmentation was performed on the Oxford-IIIT Pet dataset. The ground-truth mask was displayed, and a U-Net model was trained to predict a class for every pixel of the image.

Viva Questions

Q. Difference between classification and segmentation?
A. Classification gives one label to the whole image; segmentation gives a label to every pixel.

Q. Why does U-Net use skip connections?
A. They pass fine spatial details from the encoder to the decoder, so the mask edges are sharper.

Q. Which loss is used and why?
A. Sparse categorical cross-entropy, because each pixel belongs to one of 3 classes and the mask holds integer labels.

Experiment 8: Predict a Caption for a Sample Image Using LSTM

Aim

To generate a meaningful caption for an image using a deep learning model with an image encoder and a language decoder.

Theory

Image captioning combines Computer Vision and Natural Language Processing: the model looks at an image and writes a sentence about it.

It has two parts: an **encoder (CNN)** that extracts a feature vector from the image, and a **decoder (LSTM)** that produces the caption word by word from that vector.

Workflow: Input image, feature extraction, feature vector, caption generation, output sentence.

Applications: help for visually impaired users, automatic image tagging, social media tagging and smart photo management.

Dataset: Flickr8k: 8,000 images with 5 captions each (people, animals, sports and daily scenes).

Model used in the program: the pre-trained *nlpconnect/vit-gpt2-image-captioning* model from Hugging Face. It follows the same encoder-decoder idea: a Vision Transformer (ViT) is the image encoder and GPT-2 is the language decoder (in place of CNN and LSTM).

Software Requirements

Google Colab, Python 3.x, transformers, PyTorch, Pillow

Algorithm

1. Install the required libraries.
2. Upload an image.
3. Open the image and convert it to RGB.
4. Load the pre-trained captioning model, the image processor and the tokenizer.
5. Convert the image to pixel values.
6. Generate the caption token ids with model.generate().
7. Decode the ids to text and print the caption.

Program

```
!pip install transformers pillow torch

from transformers import VisionEncoderDecoderModel
from transformers import ViTImageProcessor
from transformers import AutoTokenizer
from PIL import Image
from google.colab import files
import torch

# Upload Image
uploaded = files.upload()
image_name = list(uploaded.keys())[0]

# Load image
image = Image.open(image_name).convert("RGB")

# Load pretrained image captioning model
model = VisionEncoderDecoderModel.from_pretrained(
    "nlpconnect/vit-gpt2-image-captioning")
feature_extractor = ViTImageProcessor.from_pretrained(
    "nlpconnect/vit-gpt2-image-captioning")
tokenizer = AutoTokenizer.from_pretrained(
    "nlpconnect/vit-gpt2-image-captioning")

# Process image
pixel_values = feature_extractor(images=image,
                                 return_tensors="pt").pixel_values

# Generate caption
output_ids = model.generate(pixel_values, max_length=20)
caption = tokenizer.decode(output_ids[0], skip_special_tokens=True)

print("Generated Caption:")
print(caption)
```

Note: The manual's theory talks about CNN + LSTM, but the program uses ViT + GPT-2 (both are encoder-decoder models). If the examiner asks for the classic CNN-LSTM form, explain it like this: a CNN such as InceptionV3 or ResNet gives a feature vector, a Dense layer reduces it, and an LSTM with an Embedding layer predicts the next word using softmax, one word at a time, until the end token.

Expected Output

```
Generated Caption:
a dog sitting on a couch with a blanket

(Example for a photo of a dog on a sofa. The caption depends on the image that you upload.)
```

Result

A caption was generated for the uploaded image using a pre-trained image encoder - language decoder model.

Viva Questions

Q. Which two networks make an image captioning model?
A. An encoder (CNN or ViT) for image features and a decoder (LSTM or GPT-2) for generating the words.

Q. Why is LSTM good for captions?
A. A caption is a sequence of words and the LSTM remembers the earlier words while predicting the next word.

Q. What does max_length=20 do?
A. It limits the caption to at most 20 tokens.

Experiment 9: Program to Detect a Dog Image Using the YOLO Algorithm

Aim

To detect and locate a dog in an image using the YOLO (You Only Look Once) object detection algorithm.

Theory

Object detection finds **what** objects are in an image and **where** they are, using bounding boxes. Classification only tells what is present.

YOLO is a real-time detector that looks at the whole image in one pass through the network, so it is much faster than older multi-stage methods.

Working of YOLO:

- The image is divided into a grid.
- The network predicts bounding boxes and class probabilities for each grid cell.
- All objects are detected in a single forward pass.
- Non-Maximum Suppression (NMS) removes duplicate boxes.
- The final objects are shown with bounding boxes and labels.

Advantages: high speed, real-time detection, good accuracy and many classes. **Dataset:** COCO (over 330,000 images, 80 classes such as dog, cat, person, car, bicycle, horse, bird and bus). YOLOv8n is pre-trained on COCO.

Software Requirements

Google Colab, Python 3.x, ultralytics (YOLOv8), Pillow, Matplotlib

Algorithm

1. Install the ultralytics package (YOLOv8).
2. Upload a dog image (dog.jpg).
3. Load the pre-trained model yolov8n.pt.
4. Run detection on the image.
5. Display the result and save it as dog_detection_result.jpg.
6. Show the saved image with Matplotlib.

Program

Step 1 and 2: Install YOLOv8 and upload the image

```
!pip install ultralytics

from google.colab import files
uploaded = files.upload()      # upload an image named dog.jpg
```

Step 3: Detect the dog

```
from ultralytics import YOLO
from PIL import Image
import matplotlib.pyplot as plt

# Load YOLOv8 pretrained model
model = YOLO("yolov8n.pt")

# Run detection
results = model("dog.jpg")

# Display result
results[0].show()

# Save output image
results[0].save(filename="dog_detection_result.jpg")
print("Dog Detection Completed Successfully")
```

Step 4: Display the output image

```
img = Image.open("dog_detection_result.jpg")
plt.figure(figsize=(8, 8))
plt.imshow(img)
plt.axis("off")
plt.show()
```

Expected Output

```
image 1/1 /content/dog.jpg: 480x640 1 dog, 70.2ms
Speed: 3.1ms preprocess, 70.2ms inference, 1.8ms postprocess per image
Dog Detection Completed Successfully

(The image shown has a coloured box around the dog with the label "dog" and a confidence such as 0.92. The numbers in
```

Result

The dog in the image was detected and located with a bounding box and the label "dog" using the pre-trained YOLOv8 model, and the output image was saved.

Viva Questions

Q. What does YOLO stand for?
A. You Only Look Once: the whole image is processed in one pass.

Q. What is Non-Maximum Suppression?
A. A step that removes overlapping boxes of the same object and keeps the one with the highest confidence.

Q. Why use a pre-trained model?
A. It is already trained on the large COCO dataset, so no training is needed and 'dog' is one of its 80 classes.

Experiment 10: Character Recognition Using RNN and Comparison with CNN

Aim

To implement character (handwritten digit) recognition using a Recurrent Neural Network and to compare its performance with a Convolutional Neural Network on the MNIST dataset.

Theory

Character recognition identifies handwritten or printed characters in images. It is a major use of deep learning and OCR.

RNN: a network for sequential data that keeps information from earlier inputs in an internal state. For an image, each of the 28 rows is treated as one time step with 28 values, so the image becomes a sequence of 28 rows.

CNN: a network for images. Convolution layers find spatial features (edges, shapes) and pooling layers reduce the size, so the CNN uses the 2-D structure of the image directly.

Dataset: MNIST: 70,000 grayscale images of the digits 0 to 9, size 28 x 28 (60,000 for training and 10,000 for testing).

Comparison of RNN and CNN:

Feature	RNN	CNN
Data type	Sequential data	Image data
Memory	Uses previous information	No internal memory
Speed	Slower	Faster
Accuracy on images	Moderate	High
Best use	Text, speech, time series	Images, videos
Typical MNIST accuracy (5 epochs)	about 97 to 98 %	about 98 to 99 %

Software Requirements

Google Colab, Python 3.x, TensorFlow/Keras

Algorithm

1. Import the libraries and load the MNIST dataset.
2. Normalize pixel values to 0 to 1 and one-hot encode the labels.
3. Part A: build a SimpleRNN(128) model with a Dense(10, softmax) output, train for 5 epochs and test it.
4. Part B: reshape the images to (28, 28, 1), build a CNN (Conv2D, MaxPooling2D, Flatten, Dense), train for 5 epochs and test it.
5. Compare the RNN accuracy with the CNN accuracy.

Program

Step 1: Import libraries and load the dataset

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, SimpleRNN
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten
from tensorflow.keras.utils import to_categorical

# Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

Part A: Character recognition using RNN

```
rnn_model = Sequential([
    Input(shape=(28, 28)),
    SimpleRNN(128),
    Dense(10, activation='softmax')
])

rnn_model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

rnn_model.fit(X_train, y_train_cat, epochs=5, batch_size=128,
              validation_split=0.1)

rnn_loss, rnn_acc = rnn_model.evaluate(X_test, y_test_cat)
print("RNN Accuracy:", rnn_acc)
```

Part B: Character recognition using CNN

```
# Reshape Data
X_train_cnn = X_train.reshape(-1, 28, 28, 1)
X_test_cnn = X_test.reshape(-1, 28, 28, 1)

cnn_model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

cnn_model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

cnn_model.fit(X_train_cnn, y_train_cat, epochs=5, batch_size=128,
              validation_split=0.1)

cnn_loss, cnn_acc = cnn_model.evaluate(X_test_cnn, y_test_cat)
print("CNN Accuracy:", cnn_acc)
```

Note: Input(shape=...) is used instead of input_shape inside the first layer (this only avoids a Keras warning).

Expected Output

```
RNN (typical):
Epoch 1/5 - accuracy: 0.86 - val_accuracy: 0.95
...
Epoch 5/5 - accuracy: 0.98 - val_accuracy: 0.97
RNN Accuracy: 0.97

CNN (typical):
Epoch 1/5 - accuracy: 0.92 - val_accuracy: 0.98
...
Epoch 5/5 - accuracy: 0.995 - val_accuracy: 0.99
CNN Accuracy: 0.99

(Typical values, not exact. The CNN is usually 1 to 2 percent more accurate than the RNN and trains faster per epoch.
```

Result

Handwritten digits of MNIST were recognized with both an RNN and a CNN. The CNN gave higher accuracy (about 98 to 99 %) than the RNN (about 97 to 98 %), so the CNN is better for image data, while the RNN is better for sequential data.

Viva Questions

Q. Why can an RNN be used on an image?
A. Each image row is treated as one time step, so the 28 x 28 image becomes a sequence of 28 vectors.

Q. Why is the CNN better here?
A. It uses the 2-D spatial pattern of pixels with shared filters, so it learns shapes with fewer parameters and trains faster.

Q. Why one-hot encoding and categorical cross-entropy?
A. The 10 digit classes become 10-value vectors, which match the 10 softmax outputs of the model.